

MECHATRONICS AND IOT

ASSIGNMENT REPORT - 10

TITLE:

Design and develop a Wireless Environmental Parameter Monitoring System for Smart Industries. The system remotely monitors environmental conditions such as temperature, humidity, and air quality in industrial environments.

DONE BY ,

HARI KRISHNA JAGADEESH-24MER011

HARISH DHARMALINGAM – 24MER014

RANJITH KUMAR P – 24MER048

RISHI KESAV A – 24MER049

VENKATRAMANAN B – 24MER061

Project Overview:

The Wireless Environmental Parameter Monitoring System is an IoT-based system designed to continuously monitor important environmental conditions in industrial environments. The system uses sensors to measure parameters such as temperature, humidity and air quality, which are processed by a microcontroller.

The collected environmental data is displayed locally on an OLED/LCD display and can be transmitted wirelessly using the microcontroller's Wi-Fi connectivity. The system can monitor the measured parameters in real time and identify abnormal environmental conditions based on predefined threshold values.

When the temperature, humidity or air-quality level exceeds the specified limits, the system can generate LED and buzzer alerts to notify industrial personnel. The monitored data can also be stored on an IoT/cloud platform for remote monitoring, data logging and analysis.

The proposed system provides a low-cost, real-time and wireless solution for monitoring industrial environmental conditions, improving workplace safety, supporting early detection of hazardous conditions and enabling better environmental management in smart industries.

Problem Statement:

Industrial environments can experience changes in temperature, humidity and air quality due to machinery, chemical processes, poor ventilation and other operating conditions. Continuous monitoring of these parameters is important for maintaining worker safety, equipment reliability and a healthy industrial environment.

Conventional monitoring methods often depend on manual inspection or periodic measurements, which may result in delayed detection of abnormal environmental conditions. This can expose workers to harmful conditions and affect industrial operations.

Therefore, there is a need for a wireless, real-time environmental monitoring system that can continuously measure temperature, humidity and air quality, provide immediate alerts when critical levels are detected, and enable remote monitoring and data analysis through IoT connectivity.

Proposed Solution:

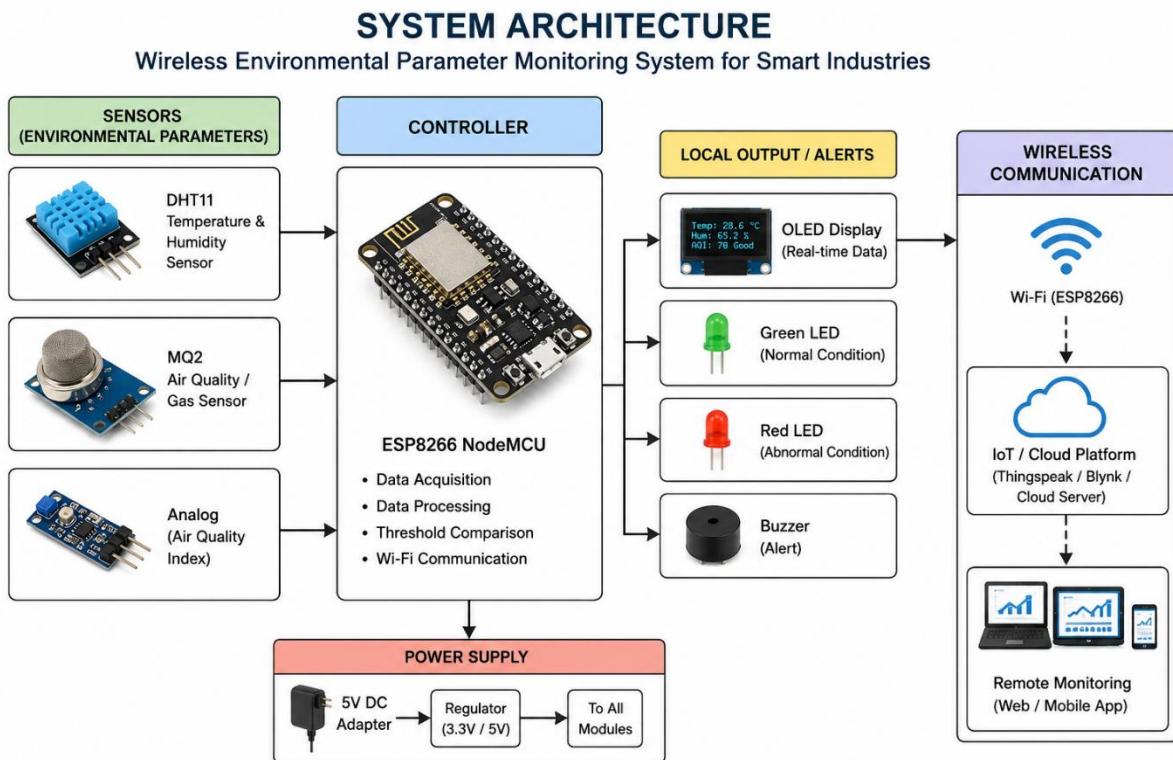
The proposed system uses an ESP8266 NodeMCU along with sensors to continuously monitor important environmental parameters such as temperature, humidity and air quality in industrial environments. A DHT11 sensor is used to measure temperature and humidity, while an MQ2 gas sensor is used to detect smoke and harmful combustible gases.

The ESP8266 collects and processes the sensor readings and displays the measured values on an OLED display for real-time local monitoring. The readings are compared with predefined threshold values to identify normal and abnormal environmental conditions.

When any parameter exceeds the specified limit, the system activates a red LED and buzzer to provide an immediate warning, while a green LED indicates normal conditions. The ESP8266's built-in Wi-Fi connectivity can transmit the environmental data to an IoT/cloud platform for remote monitoring, storage and analysis.

The proposed solution provides a low-cost, wireless and real-time environmental monitoring system that can improve workplace safety, detect hazardous conditions early and support efficient environmental management in smart industries.

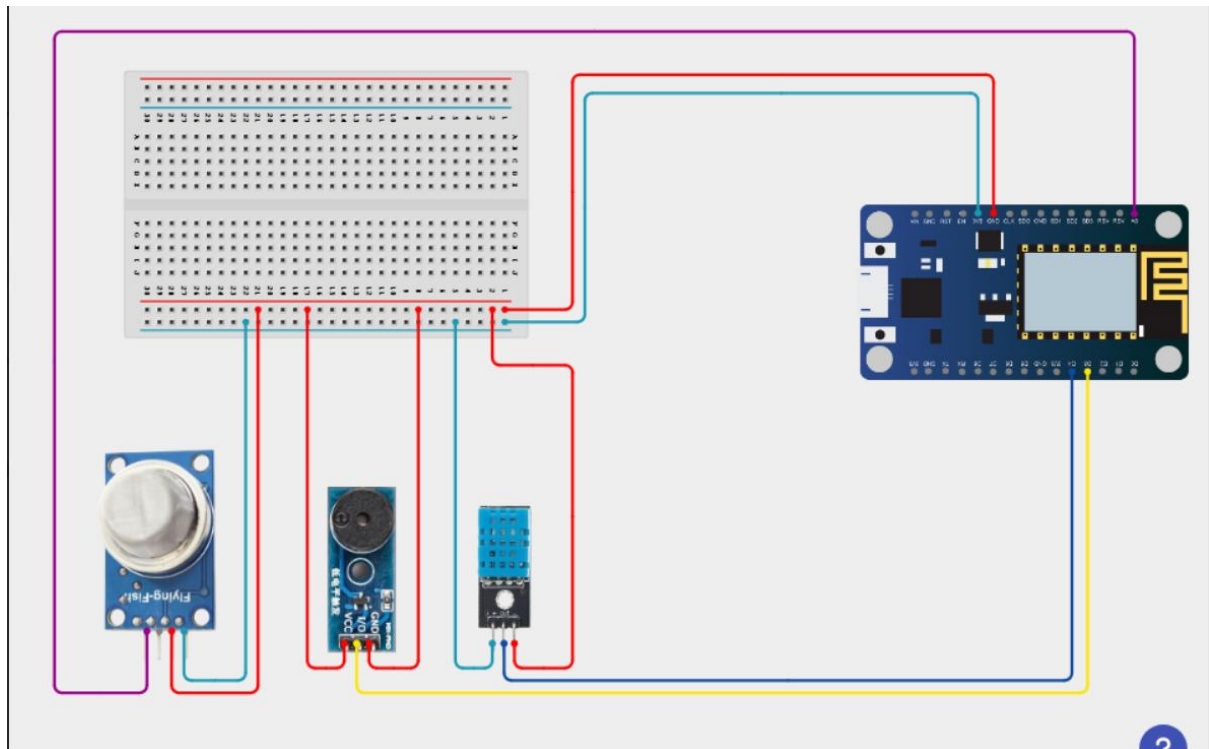
System Architecture:



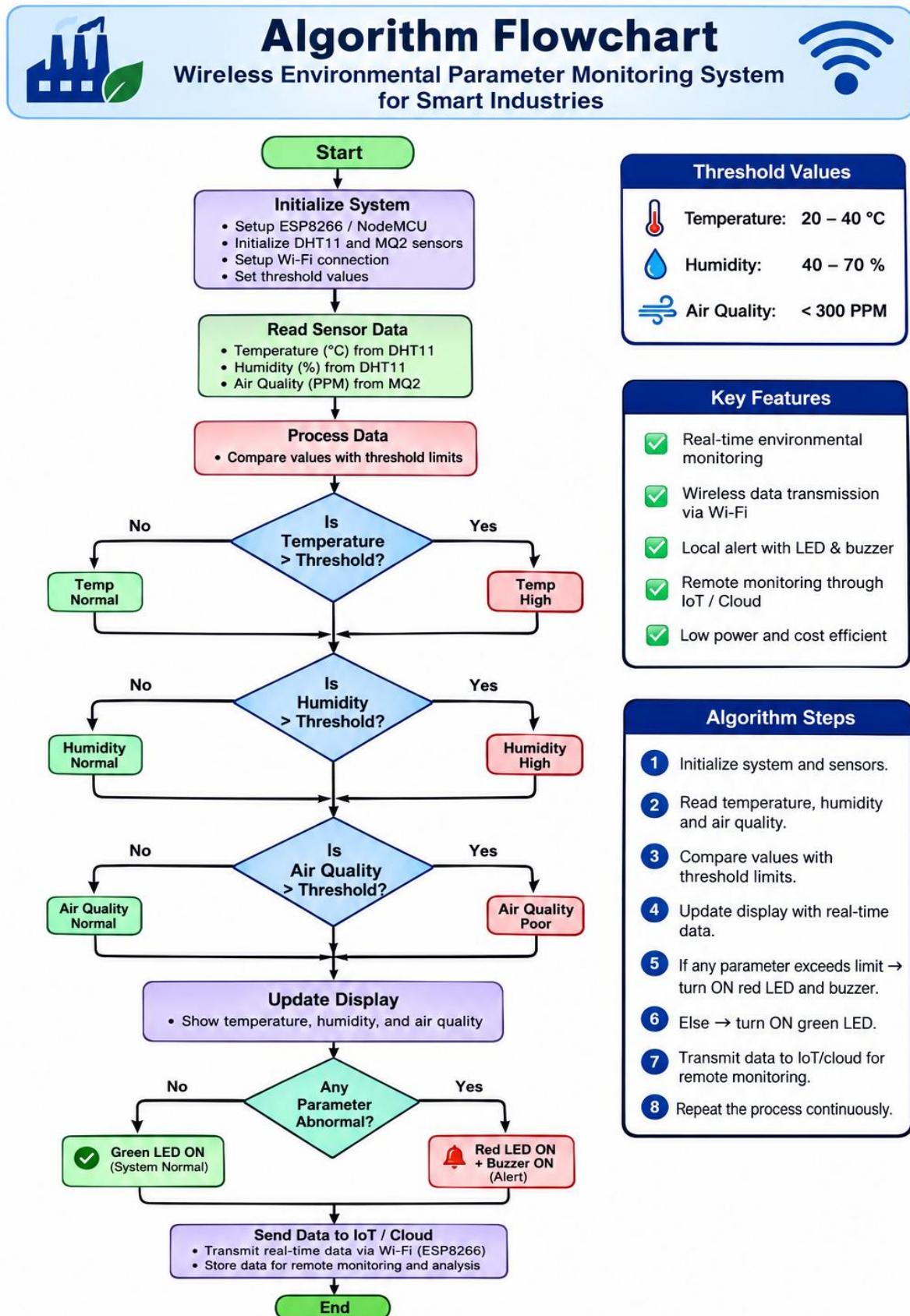
Circuit Table :

Component	Pin/Terminal	ESP8266 Connection	Purpose
DHT11 Sensor	VCC	3.3V	Power supply
LCD Display	VCC	3.3V	Safe indication
ESP8266	VIN/USB	5V USB	Hazardous indication
Buzzer	Positive (+)	GPIO 14	Alert
Green LED	Anode (+)	D5 (GPIO 14) through 220 Ω	Normal condition
MQ2 Gas Sensor	A0	A0 through voltage divider	Air-quality/gas-level reading

Circuit Design:



Algorithm:



Coding:

```
#include <ESP8266WiFi.h>

#include "DHT.h"

#include "Adafruit_MQTT.h"

#include "Adafruit_MQTT_Client.h"

// =====

// WIFI SETTINGS

// =====

#define WIFI_SSID    "Hari"

#define WIFI_PASSWORD "11111111"

// =====

// ADAFRUIT IO SETTINGS

// =====

#define AIO_SERVER    "io.adafruit.com"

#define AIO_SERVERPORT 1883

#define AIO_USERNAME  "RISHI_44"

// IMPORTANT:

// Put your NEW regenerated Adafruit IO key here

#define AIO_KEY      "aio_lmgi430fO7kh0Cq5iCaE2r1cGwPy"

// =====

// PIN CONFIGURATION

// =====

// D4 = GPIO2

#define DHTPIN 2

// MQ-135 analog output

#define MQ135_PIN A0

// D5 = GPIO14

#define BUZZER_PIN 14

// D6 = GPIO12
```

```
#define LED_PIN 12

// =====

// DHT SETTINGS

// =====

#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// =====

// WIFI CLIENT

// =====

WiFiClient client;

// =====

// ADAFRUIT MQTT CLIENT

// =====

Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);

// =====

// ADAFRUIT IO FEEDS

// =====

Adafruit_MQTT_Publish temperatureFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/temperature"
  );

Adafruit_MQTT_Publish humidityFeed =
  Adafruit_MQTT_Publish(
```



```

    &mqtt,
    AIO_USERNAME "/feeds/humidity"
);
Adafruit_MQTT_Publish airQualityFeed =
Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/air-quality"
);
Adafruit_MQTT_Publish alertFeed =
Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/alert"
);
// =====
// AIR QUALITY LIMIT
// =====
const int AIR_QUALITY_LIMIT = 300;
// =====
// WIFI CONNECTION
// =====

void connectWiFi()
{
    if (WiFi.status() == WL_CONNECTED)
    {
        return;
    }
    Serial.print("Connecting to WiFi");
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    while (WiFi.status() != WL_CONNECTED)
    {

```

```
    delay(500);

    Serial.print(".");
}

Serial.println();

Serial.println("WiFi Connected!");

Serial.print("IP Address: ");

Serial.println(WiFi.localIP());

Serial.print("Signal Strength: ");

Serial.print(WiFi.RSSI());

Serial.println(" dBm");
}

// =====

// ADAFRUIT IO CONNECTION

// =====

void connectMQTT()
{
    int8_t ret;

    if (mqtt.connected())
    {
        return;
    }

    Serial.print("Connecting to Adafruit IO");

    while ((ret = mqtt.connect()) != 0)
    {
        Serial.print(".");

        mqtt.disconnect();

        delay(5000);
    }

    Serial.println();

    Serial.println("Adafruit IO Connected!");
```

```
}  
  
// =====  
  
// SETUP  
  
// =====  
  
void setup()  
{  
  Serial.begin(115200);  
  delay(1000);  
  Serial.println();  
  Serial.println("=====");  
  Serial.println(" WIRELESS ENVIRONMENTAL MONITORING SYSTEM");  
  Serial.println("=====");  
  // =====  
  // OUTPUT PINS  
  // =====  
  pinMode(BUZZER_PIN, OUTPUT);  
  pinMode(LED_PIN, OUTPUT);  
  // =====  
  // ACTIVE-LOW BUZZER  
  // HIGH = OFF  
  // LOW = ON  
  // =====  
  digitalWrite(BUZZER_PIN, HIGH);  
  // LED OFF  
  digitalWrite(LED_PIN, LOW);  
  // =====  
  // START DHT11  
  // =====  
  dht.begin();  
  Serial.println("DHT11 initialized.");
```

```
// =====  
  
// CONNECT WIFI  
  
// =====  
  
connectWiFi();  
  
// =====  
  
// CONNECT ADAFRUIT IO  
  
// =====  
  
connectMQTT();  
Serial.println();  
Serial.println("SYSTEM READY!");  
Serial.println();  
}  
  
// =====  
  
// MAIN LOOP  
  
// =====  
  
void loop()  
{  
  
// =====  
  
// CHECK WIFI  
  
// =====  
  
if (WiFi.status() != WL_CONNECTED)  
{  
    Serial.println("WiFi disconnected!");  
    connectWiFi();  
}  
  
// =====  
  
// CHECK ADAFRUIT IO  
  
// =====  
  
connectMQTT();
```

```

// =====
// READ DHT11
// =====

float temperature = dht.readTemperature();
float humidity = dht.readHumidity();

// =====
// CHECK DHT11
// =====

if (isnan(temperature) || isnan(humidity))
{
    Serial.println("ERROR: DHT11 read failed!");
    delay(2000);
    return;
}

// =====
// READ MQ-135
// =====

int airQuality = analogRead(MQ135_PIN);

// =====
// PRINT SENSOR VALUES
// =====

Serial.println();
Serial.println("=====");
Serial.print("Temperature : ");
Serial.print(temperature);
Serial.println(" °C");
Serial.print("Humidity   : ");
Serial.print(humidity);
Serial.println(" %");
Serial.print("Air Quality : ");

```

```
Serial.println(airQuality);

Serial.print("Air Limit  : ");

Serial.println(AIR_QUALITY_LIMIT);

// =====

// AIR QUALITY STATUS

// =====

bool alert = false;

if (airQuality > AIR_QUALITY_LIMIT)

{

    alert = true;

}

// =====

// BUZZER + LED CONTROL

// =====

if (alert)

{

    // Active LOW buzzer

    digitalWrite(BUZZER_PIN, LOW);

    // LED ON

    digitalWrite(LED_PIN, HIGH);

    Serial.println();

    Serial.println("*****");

    Serial.println("    AIR QUALITY ALERT");

    Serial.println("*****");

    Serial.println("STATUS : ALERT");

    Serial.println("BUZZER : ON");

    Serial.println("LED   : ON");

}

else

{
```

```
// Active LOW buzzer OFF

digitalWrite(BUZZER_PIN, HIGH);

// LED OFF

digitalWrite(LED_PIN, LOW);

Serial.println();

Serial.println("STATUS : NORMAL");

Serial.println("BUZZER : OFF");

Serial.println("LED   : OFF");

}

// =====

// SEND TEMPERATURE

// =====

Serial.println();

Serial.print("Temperature upload : ");

if (temperatureFeed.publish(temperature))

{

    Serial.println("SUCCESS");

}

else

{

    Serial.println("FAILED");

}

// =====

// SEND HUMIDITY

// =====

Serial.print("Humidity upload   : ");

if (humidityFeed.publish(humidity))

{

    Serial.println("SUCCESS");

}
```

```
else

{
    Serial.println("FAILED");
}

// =====

// SEND AIR QUALITY

// =====

Serial.print("Air quality upload : ");
if (airQualityFeed.publish(airQuality))
{
    Serial.println("SUCCESS");
}
else
{
    Serial.println("FAILED");
}

// =====

// SEND STATUS AS TEXT

// =====

Serial.print("Status upload    : ");
if (alert)
{
    if (alertFeed.publish("ALERT"))
    {
        Serial.println("ALERT");
    }
    else
    {
        Serial.println("FAILED");
    }
}
```



```
}  
else  
{  
  if (alertFeed.publish("NORMAL"))  
  {  
    Serial.println("NORMAL");  
  }  
  else  
  {  
    Serial.println("FAILED");  
  }  
}  
  
// =====  
// FINAL STATUS  
// =====  
Serial.println("-----");  
if (alert)  
{  
  Serial.println("FINAL STATUS: ALERT");  
}  
else  
{  
  Serial.println("FINAL STATUS: NORMAL");  
}  
Serial.println("Next update in 20 seconds...");  
Serial.println("=====");  
// =====  
// WAIT  
// =====  
delay(20000) }
```

System Working:

- The Wireless Environmental Parameter Monitoring System uses an ESP8266 NodeMCU, DHT11 sensor and MQ2 gas sensor to continuously monitor environmental conditions in an industrial environment. The DHT11 measures temperature and humidity, while the MQ2 sensor detects smoke and harmful gases.
- The sensors send their readings to the ESP8266, which processes the data and compares the values with predefined threshold limits. The measured parameters are displayed on an OLED display for real-time monitoring.
- When all parameters are within the acceptable range, the green LED indicates normal conditions. If the temperature, humidity or air-quality level exceeds the specified limit, the red LED and buzzer are activated to provide an immediate warning.
- The ESP8266 uses its built-in Wi-Fi connectivity to transmit the environmental data to an IoT/cloud platform for remote monitoring, data storage and analysis. The system continuously repeats the monitoring process, enabling early detection of hazardous conditions and improved safety in smart industrial environments.

Bill Of Material:

S. no	Component	Specification	Quantity	Cost
1	ESP8266	Node MCU ESP8266	1	Rs. 450
2	DHT11 Sensor	Temperature & Humidity Sensor	1	Rs. 100
3	MQ2 Gas Sensor	Smoke & Gas Detection	1	Rs. 100
4	Bread Board	Standard 830-point	1	Rs. 100
5	Jumper Wire	Male-Male / Male-Female	1 set	Rs. 50
6	OLED Display	0.96" I ² C, 128×64	1	Rs. 150
		Total		Rs. 950

Prototype Implementation:

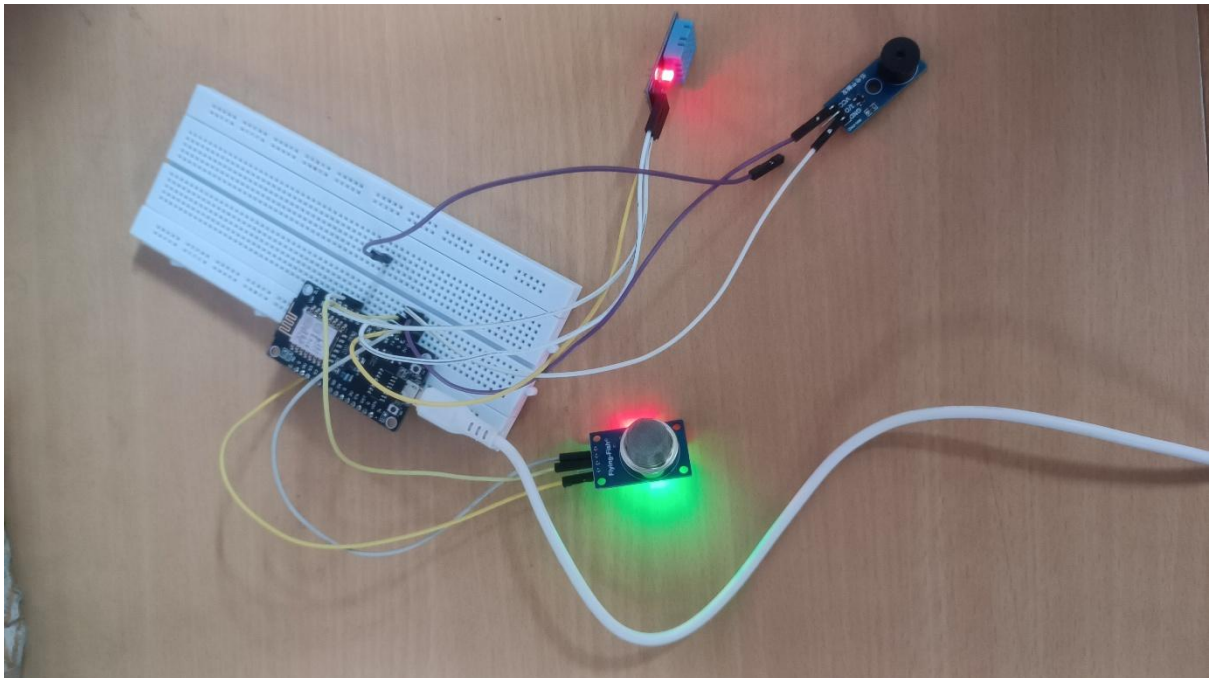
- The prototype was implemented using an ESP8266 NodeMCU, DHT11 sensor, MQ2 gas sensor, OLED display, green LED, red LED and buzzer. The DHT11 sensor was used to measure the temperature and humidity, while the MQ2 sensor was used to detect smoke and harmful gases in the surrounding environment.
- The ESP8266 collects the sensor readings and processes them in real time. The measured temperature, humidity and air-quality information is displayed on the OLED display for local monitoring. The readings are compared with predefined threshold values to determine the environmental condition.
- During normal conditions, the green LED indicates that the environment is within the acceptable range. If any monitored parameter exceeds the specified limit, the red LED and buzzer are activated to provide an immediate warning.
- The ESP8266's built-in Wi-Fi connectivity can also be used to transmit the environmental data to an IoT/cloud platform for remote monitoring and data logging.

Prototype

flow:

**DHT11 + MQ2 → ESP8266 → Data Processing → OLED Display + LED/Buzzer
Alert → Wi-Fi → Cloud Monitoring**

The prototype demonstrates a **low-cost, wireless and real-time environmental monitoring solution** suitable for smart industrial applications.



Results and Performance:

- The developed Wireless Environmental Parameter Monitoring System successfully monitored temperature, humidity and air quality using the DHT11 and MQ2 sensors connected to the ESP8266. The sensor readings were processed in real time and displayed on the OLED display.
- The system successfully compared the measured values with predefined threshold limits. Under normal environmental conditions, the green LED indicated safe operation. When the temperature, humidity or air-quality level exceeded the specified limit, the red LED and buzzer were activated to provide an immediate warning.

Performance Analysis

Parameter	Result
Temperature Monitoring	Successfully measured using DHT11
Humidity Monitoring	Successfully measured using DHT11
Air Quality Monitoring	Successfully detected using MQ2
Local Monitoring	Real-time OLED display
Normal Condition	Green LED ON
Abnormal Condition	Red LED + Buzzer ON
Response	Real-time
Wireless Connectivity	ESP8266 Wi-Fi
Remote Monitoring	IoT/Cloud integration supported

Overall, the prototype demonstrated a low-cost, wireless and real-time environmental monitoring solution. Continuous monitoring helps detect abnormal environmental conditions early, improving workplace safety, environmental management and operational reliability in smart industrial environments.

